

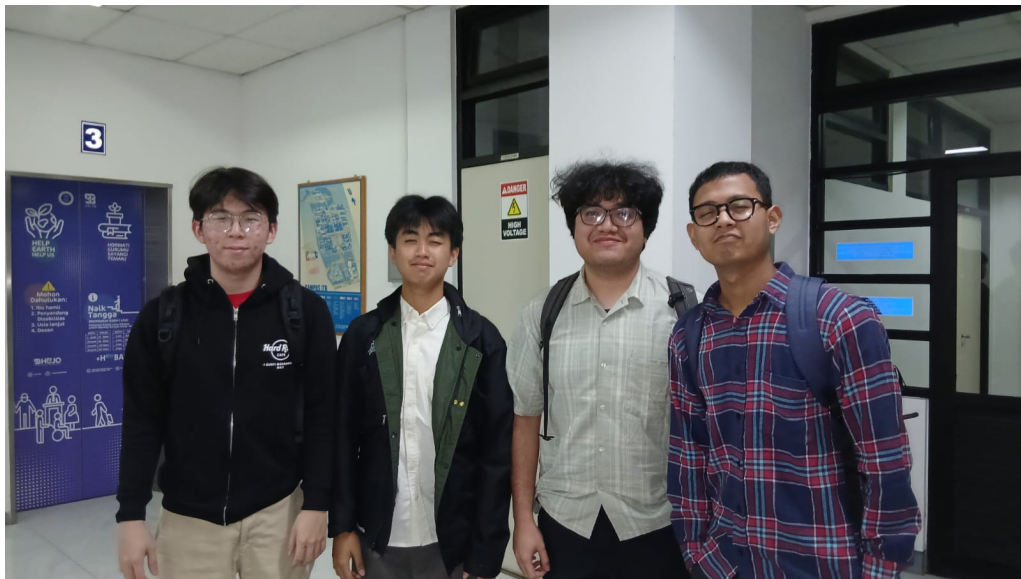
# Tugas Besar

## Milestone 3

IF2224 Teori Bahasa Formal dan Otomata

Semantic Analysis

Semester 2 Tahun 2025/2026



**Disusun oleh:**

**Kai**

|                             |            |
|-----------------------------|------------|
| Jonathan Kris Wicaksono     | (13524023) |
| Vincent Rionarlie           | (13524031) |
| Muhammad Aufar Rizqi Kusuma | (13524061) |
| Bryan Pratama Putra Hendra  | (13524067) |

Laboratorium Ilmu dan Rekayasa Komputasi  
Program Studi Teknik Informatika  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

# Daftar Isi

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Pendahuluan</b>                       | <b>3</b>  |
| 1.1      | Latar Belakang . . . . .                 | 3         |
| 1.2      | Rumusan Masalah . . . . .                | 3         |
| 1.3      | Tujuan . . . . .                         | 4         |
| 1.4      | Batasan Masalah . . . . .                | 4         |
| <b>2</b> | <b>Dasar Teori</b>                       | <b>5</b>  |
| 2.1      | Semantic Analysis . . . . .              | 5         |
| 2.2      | Abstract Syntax Tree . . . . .           | 5         |
| 2.3      | Decorated AST . . . . .                  | 6         |
| 2.4      | Symbol Table . . . . .                   | 6         |
| 2.4.1    | tab . . . . .                            | 7         |
| 2.4.2    | btab . . . . .                           | 7         |
| 2.4.3    | atab . . . . .                           | 7         |
| 2.5      | Type Compatibility . . . . .             | 7         |
| 2.6      | Predefined Identifier . . . . .          | 8         |
| <b>3</b> | <b>Perancangan dan Implementasi</b>      | <b>9</b>  |
| 3.1      | Gambaran Umum Program . . . . .          | 9         |
| 3.2      | Struktur Direktori Program . . . . .     | 9         |
| 3.3      | Perancangan AST . . . . .                | 10        |
| 3.4      | Syntax-Directed Translation . . . . .    | 10        |
| 3.5      | Implementasi Symbol Table . . . . .      | 11        |
| 3.6      | Implementasi Semantic Analyzer . . . . . | 11        |
| 3.6.1    | Scope dan Block . . . . .                | 12        |
| 3.6.2    | Type Resolution . . . . .                | 12        |
| 3.6.3    | Type Checking . . . . .                  | 12        |
| 3.6.4    | Procedure dan Function . . . . .         | 12        |
| 3.7      | Format Output . . . . .                  | 12        |
| <b>4</b> | <b>Pengujian</b>                         | <b>13</b> |
| 4.1      | Strategi Pengujian . . . . .             | 13        |
| 4.2      | Detail Pengujian . . . . .               | 14        |
| 4.2.1    | Kasus 1: Program Dasar . . . . .         | 14        |
| 4.2.2    | Kasus 2: Ekspresi dan Boolean . . . . .  | 15        |
| 4.2.3    | Kasus 3: Control Flow . . . . .          | 16        |
| 4.2.4    | Kasus 4: Structured Type . . . . .       | 17        |



|          |                                    |           |
|----------|------------------------------------|-----------|
| 4.2.5    | Kasus 5: Subprogram . . . . .      | 18        |
| 4.3      | Pengujian Error Semantic . . . . . | 20        |
| <b>5</b> | <b>Penutup</b>                     | <b>21</b> |
| 5.1      | Kesimpulan . . . . .               | 21        |
| 5.2      | Saran . . . . .                    | 21        |
| <b>6</b> | <b>Lampiran</b>                    | <b>22</b> |
| 6.1      | Tautan GitHub . . . . .            | 22        |
| 6.2      | Tabel Kontribusi . . . . .         | 23        |
| 6.3      | Cara Menjalankan Program . . . . . | 23        |

# Bab 1

## Pendahuluan

### 1.1 Latar Belakang

Compiler merupakan perangkat lunak yang menerjemahkan program dari bahasa tingkat tinggi menjadi representasi yang dapat diproses lebih lanjut oleh mesin. Tahap awal compiler tidak langsung menghasilkan kode mesin, melainkan membentuk representasi internal yang semakin kaya. Lexical analysis mengubah karakter menjadi token, syntax analysis memeriksa struktur token berdasarkan grammar, dan semantic analysis memeriksa makna program yang tidak dapat ditentukan hanya dari grammar.

Pada Milestone 1, Arion Compiler telah memiliki lexer yang mampu menghasilkan token stream. Pada Milestone 2, token tersebut diproses oleh Recursive Descent Parser untuk menghasilkan parse tree. Parse tree menunjukkan bahwa program valid secara sintaksis, tetapi belum menjamin bahwa program bermakna benar. Program seperti assignment string ke integer, pemakaian identifier yang belum dideklarasikan, atau pemanggilan fungsi dengan parameter yang salah dapat saja memiliki bentuk sintaks yang valid, tetapi tetap salah secara semantik.

Milestone 3 berfokus pada semantic analysis. Tahap ini mengubah parse tree menjadi Abstract Syntax Tree (AST), mengelola symbol table, melakukan type checking, mengatur scope, dan memberi anotasi pada AST sehingga setiap node memiliki informasi semantik seperti tipe data, lexical level, block index, dan referensi ke symbol table. Hasil akhirnya adalah decorated AST beserta tiga tabel simbol, yaitu `tab`, `btabs`, dan `atab`.

### 1.2 Rumusan Masalah

Permasalahan utama pada Milestone 3 adalah bagaimana membangun semantic analyzer yang dapat memanfaatkan parse tree hasil Milestone 2 untuk menghasilkan representasi yang lebih ringkas dan bermakna. Permasalahan tersebut diuraikan menjadi beberapa pertanyaan berikut.

1. Bagaimana mengonversi parse tree menjadi AST menggunakan prinsip Syntax-Directed Translation?

2. Bagaimana merancang struktur AST yang cukup ekspresif untuk merepresentasikan deklarasi, statement, ekspresi, procedure, function, array, record, dan control flow?
3. Bagaimana mengelola symbol table `tab`, `btab`, dan `atab` untuk menyimpan identifier, block, scope, dan informasi array?
4. Bagaimana melakukan lookup identifier, pengecekan redeklarasasi, type checking, assignment compatibility, dan validasi control flow?
5. Bagaimana menampilkan output semantic analysis dalam bentuk symbol table dan decorated AST yang mudah diperiksa?

### 1.3 Tujuan

Tujuan pengerjaan Milestone 3 adalah membangun tahap semantic analysis pada Arion Compiler. Secara khusus, tujuan yang ingin dicapai adalah sebagai berikut.

1. Membuat AST dari parse tree hasil parser.
2. Menghilangkan detail sintaksis yang tidak diperlukan pada tahap semantik, seperti keyword struktural dan tanda baca.
3. Mengimplementasikan symbol table `tab`, `btab`, dan `atab`.
4. Menginisialisasi predefined identifier seperti `Integer`, `Real`, `Char`, `Boolean`, `String`, `True`, `False`, `writeln`, dan `readln`.
5. Melakukan pengecekan deklarasi, scope, tipe ekspresi, assignment, parameter procedure/function, array access, record field access, serta kondisi pada statement kontrol.
6. Menghasilkan decorated AST dan symbol table sebagai output program.

### 1.4 Batasan Masalah

Implementasi Milestone 3 dibatasi pada frontend compiler Arion. Program tidak melakukan code generation atau eksekusi program Arion. Semantic analyzer bekerja pada AST yang dibangun dari parse tree hasil parser, sehingga validitas sintaks diasumsikan sudah diperiksa oleh tahap sebelumnya.

Pengecekan semantik yang diimplementasikan berfokus pada deklarasi identifier, scope, type checking, assignment compatibility, procedure/function call, array, record, subrange, enumerated type, dan validasi kondisi control flow. Fitur runtime seperti nilai aktual variabel saat program berjalan tidak dievaluasi secara penuh. Program juga belum melakukan analisis data-flow lengkap untuk mendeteksi seluruh kemungkinan penggunaan variabel sebelum inisialisasi.

## Bab 2

# Dasar Teori

### 2.1 Semantic Analysis

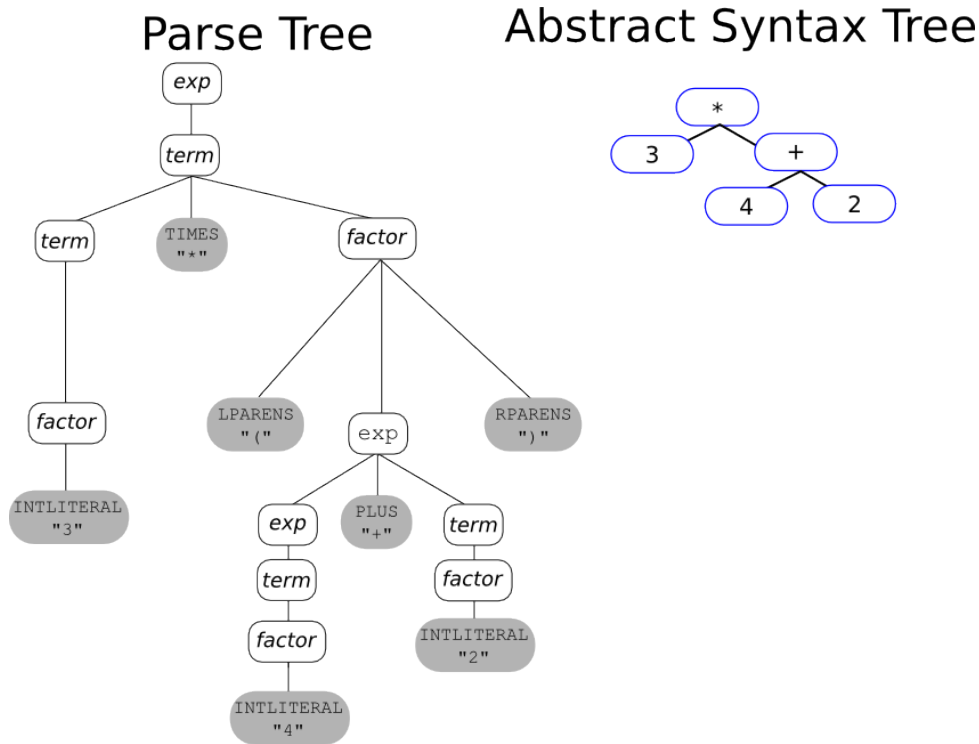
Semantic analysis adalah tahap compiler yang memeriksa makna program setelah struktur sintaksnya valid. Pada tahap ini, compiler tidak lagi hanya menanyakan apakah token tersusun sesuai grammar, tetapi juga apakah setiap konstruksi program memenuhi aturan konteks bahasa. Contohnya adalah memastikan identifikasi sudah dideklarasikan sebelum dipakai, operand operator memiliki tipe yang sesuai, assignment dilakukan ke target yang dapat diubah, dan ekspresi kondisi pada `if`, `while`, atau `repeat` bertipe Boolean [1, 3, 4].

Semantic analysis biasanya dilakukan dengan traversal terhadap tree hasil parser. Pada implementasi ini, parse tree terlebih dahulu dikonversi menjadi AST agar traversal semantik lebih sederhana. Setelah itu, semantic analyzer mengunjungi setiap node AST, menghitung tipe, melakukan lookup ke symbol table, dan menambahkan anotasi pada node [2, 5].

### 2.2 Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah representasi pohon yang lebih ringkas daripada parse tree. Parse tree memuat detail grammar seperti non-terminal perantara, tanda baca, dan keyword struktural. AST hanya menyimpan konstruksi yang penting secara semantik, misalnya assignment, binary operation, variable reference, procedure call, array access, record field access, dan control flow [5, 2].

Sebagai contoh, parse tree untuk statement `b := a + 10;` berisi node `<assignment-statement>`, `<expression>`, `<simple-expression>`, `<term>`, dan `<factor>`. Pada AST, struktur tersebut diringkas menjadi node `Assign` dengan target `Var('b')` dan value `BinOp('+')` yang memiliki anak `Var('a')` dan `Num(10)`.



Gambar 2.1: Perbandingan parse tree dan AST.

## 2.3 Decorated AST

Decorated AST adalah AST yang sudah diberi informasi tambahan hasil semantic analysis. Informasi tambahan yang digunakan pada implementasi ini meliputi:

- tipe data node, misalnya `Integer`, `Real`, `Boolean`, `Array`, atau `Record`;
- index ke `tab` untuk identifer yang berhasil ditemukan;
- lexical level tempat node dianalisis;
- block index yang mengacu pada `btabs`.

Dengan anotasi tersebut, AST tidak hanya menunjukkan struktur program, tetapi juga hasil validasi makna program. Decorated AST menjadi dasar yang lebih siap untuk tahap lanjutan seperti intermediate representation atau code generation [3, 2].

## 2.4 Symbol Table

Symbol table adalah struktur data yang menyimpan informasi identifer dan scope. Pada Milestone 3, implementasi menggunakan tiga tabel utama sesuai spesifikasi [1, 4, 6].

### 2.4.1 `tab`

`tab` menyimpan identifier seperti nama program, konstanta, variabel, tipe, procedure, dan function. Setiap entri menyimpan nama identifier, class object, tipe, referensi ke tabel lain, normal/reference flag, lexical level, address, dan link ke identifier sebelumnya dalam scope yang sama.

### 2.4.2 `btab`

`btab` menyimpan informasi block. Entry block berisi pointer ke identifier terakhir, pointer ke parameter terakhir, ukuran parameter, dan ukuran variabel lokal. Pada implementasi ini, block dibuat untuk global scope, main compound block, procedure/function, dan record type.

### 2.4.3 `atab`

`atab` menyimpan informasi array, seperti tipe index, tipe elemen, batas bawah, batas atas, ukuran elemen, dan total ukuran array. Jika sebuah tipe array dideklarasikan, semantic analyzer menambahkan informasi array tersebut ke `atab` dan menyimpan referensinya pada `tab`.

## 2.5 Type Compatibility

Type compatibility menentukan apakah dua tipe boleh digunakan bersama dalam ekspresi atau assignment. Implementasi menggunakan aturan berikut [3, 6].

1. Dua tipe kompatibel jika jenis dan referensinya sama.
2. Subrange kompatibel dengan base type-nya, misalnya subrange integer dengan integer.
3. Assignment dari Integer ke Real diperbolehkan.
4. Assignment ke subrange diperbolehkan jika tipe kompatibel dan literal berada di dalam batas subrange.
5. Operator aritmatika hanya menerima operand numerik.
6. Operator `div` dan `mod` hanya menerima operand integer-like.
7. Operator `and`, `or`, dan `not` hanya menerima Boolean.
8. Operator relasional menghasilkan Boolean dan membutuhkan operand yang kompatibel.
9. Kondisi pada `if`, `while`, dan `repeat-until` harus bertipe Boolean.





## 2.6 Predefined Identifier

Sebelum analisis program dimulai, symbol table diisi dengan predefined identifier. Identifier bawaan yang digunakan adalah `Integer`, `Real`, `Char`, `Boolean`, `String`, `True`, `False`, `writeln`, dan `readln`. Identifier `writeln` dan `readln` diperlakukan sebagai procedure bawaan dengan jumlah dan tipe argumen fleksibel [6, 1].

## Bab 3

# Perancangan dan Implementasi

### 3.1 Gambaran Umum Program

Implementasi Milestone 3 melanjutkan pipeline Milestone 1 dan Milestone 2. Alur utama berada pada `src/main.cpp`. Program membaca file input, lalu menentukan bentuk input tersebut. Jika input adalah parse tree, program menggunakan `ParseTreeReader`. Jika input adalah token stream, program menggunakan `TokenStreamReader` lalu menjalankan parser. Jika input adalah source code Arion, program menjalankan lexer dan parser.

Setelah parse tree tersedia, program menjalankan `ASTBuilder` untuk membentuk AST. AST tersebut kemudian dianalisis oleh `SemanticAnalyzer`. Jika tidak ada error, program mencetak `tab`, `btabs`, `atab`, parse tree, dan decorated AST.

#### Pipeline Program

```
Input file
-> Lexer / TokenStreamReader / ParseTreeReader
-> Parser
-> Parse Tree
-> ASTBuilder
-> AST
-> SemanticAnalyzer
-> Symbol Table + Decorated AST
```

### 3.2 Struktur Direktori Program

Bagian Milestone 3 terutama berada pada direktori `src/semantic/`. Berkas utama yang digunakan adalah sebagai berikut.

- `ast_node.h` dan `ast_node.cpp`: definisi hierarchy node AST dan struktur anotasi.
- `ast_builder.h` dan `ast_builder.cpp`: konversi parse tree menjadi AST dengan Syntax-Directed Translation.

- `ast_formatter.h` dan `ast_formatter.cpp`: pencetakan decorated AST.
- `symbol_table.h` dan `symbol_table.cpp`: implementasi `tab`, `btabs`, `atab`, `scope stack`, registrasi identifikasi, dan lookup.
- `semantic_analyzer.h` dan `semantic_analyzer.cpp`: traversal AST, type checking, scope checking, dan anotasi node.
- `parse_tree_reader.h` dan `parse_tree_reader.cpp`: pembacaan parse tree hasil Milestone 2 sebagai input Milestone 3.

### 3.3 Perancangan AST

AST dirancang sebagai hierarchy class berbasis polymorphism. Semua node mewarisi `ASTNode`, yang memiliki field `annotation`. Node AST yang digunakan mencakup `ProgramNode`, `VarDeclNode`, `ConstDeclNode`, `TypeDeclNode`, `ProcedureDeclNode`, `FunctionDeclNode`, `AssignNode`, `BinOpNode`, `UnaryOpNode`, `VarRefNode`, `ArrayAccessNode`, `FieldAccessNode`, `ProcCallNode`, `FuncCallNode`, `IfNode`, `WhileNode`, `RepeatNode`, `ForNode`, dan `CaseNode`.

Struktur anotasi pada node AST menyimpan `typeCode`, `typeName`, `tabIndex`, `blockIndex`, `level`, dan status inisialisasi. Informasi ini diisi oleh semantic analyzer setelah AST selesai dibangun.

### 3.4 Syntax-Directed Translation

Konversi parse tree ke AST dilakukan oleh `ASTBuilder`. Setiap fungsi `visit` pada builder menerima node parse tree tertentu dan mengembalikan node AST yang relevan. Detail sintaksis yang tidak diperlukan, seperti `beginsy`, `endsy`, `semicolon`, dan node grammar perantara, tidak dimasukkan ke AST.

Tabel berikut merangkum sebagian semantic action yang digunakan.

Tabel 3.1: Contoh Semantic Action ASTBuilder

| Produksi                                  | Semantic Action                                                                                                                                  |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;program&gt;</code>              | Membentuk <code>ProgramNode</code> , mengisi nama program dari header, daftar deklarasi dari declaration part, dan body dari compound statement. |
| <code>&lt;var-declaration&gt;</code>      | Untuk setiap identifier pada identifier-list, membentuk satu <code>VarDeclNode</code> dengan type specification yang sama.                       |
| <code>&lt;assignment-statement&gt;</code> | Membentuk <code>AssignNode(target, value)</code> dari node variable dan expression.                                                              |
| <code>&lt;expression&gt;</code>           | Jika terdapat relational operator, membentuk <code>BinOpNode</code> ; jika tidak, meneruskan hasil simple-expression.                            |

| Produksi            | Semantic Action                                                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <simple-expression> | Membentuk rantai BinOpNode untuk operator aditif dan UnaryOpNode untuk tanda minus di depan.                                                                    |
| <factor>            | Literal menjadi node literal, identifier menjadi VarRefNode, procedure/function call dalam expression menjadi FuncCallNode, dan not factor menjadi UnaryOpNode. |
| <array-type>        | Membentuk ArrayTypeNode berisi index type dan element type.                                                                                                     |
| <record-type>       | Membentuk RecordTypeNode yang menyimpan field sebagai daftar VarDeclNode.                                                                                       |

### 3.5 Implementasi Symbol Table

Symbol table diimplementasikan pada class `Semantic::SymbolTable`. Constructor symbol table membuat base scope dan mengisi predefined identifier. Scope dikelola dengan stack yang menyimpan `scopeLastStack` dan `scopeBlockStack`. Ketika masuk scope baru, program menambah entri `btab`; ketika keluar scope, stack dipop.

Registrasi identifier dilakukan oleh `registerIdentifier`. Fungsi ini mengecek redeklarasasi pada scope aktif, membuat entri `tab`, mengisi `link` ke identifier sebelumnya dalam scope yang sama, memperbarui `last` pada `btab`, dan menghitung ukuran parameter atau variabel lokal. Lookup dilakukan dari scope terdalam ke scope terluar.

#### Pseudocode Register Identifier

```
function registerIdentifier(name, obj, type, nrm, adr):  
    if name exists in current scope:  
        report redeclaration error  
    entry.identifier = name  
    entry.link = currentScopeLast  
    entry.obj = obj  
    entry.type = type  
    entry.lev = currentLevel  
    entry.adr = computed address  
    append entry to tab  
    current btab.last = new index
```

### 3.6 Implementasi Semantic Analyzer

Semantic analyzer melakukan traversal depth-first terhadap AST. Fungsi utama `analyze` menginisialisasi ulang symbol table, predefined procedure, struktur pendukung subrange, enum, record field, signature procedure/function, dan stack function aktif. Setelah itu `visitProgram` memproses deklarasi dan body program.

Setiap kategori node memiliki fungsi `visit`. Deklarasi diproses oleh `visitConstDecl`, `visitTypeDecl`, `visitVarDecl`, `visitProcedureDecl`, dan `visitFunctionDecl`. Statement diproses oleh `visitAssign`, `visitIf`, `visitWhile`, `visitRepeat`, `visitFor`, `visitCase`, dan `visitProcCall`. Ekspresi diproses oleh `visitExpression`, `visitVariable`, `visitConstant`, dan `visitFuncCall`.

### 3.6.1 Scope dan Block

Scope global dibuat saat symbol table diinisialisasi. Program didaftarkan sebagai object class `PROGRAM`. Main compound block dianalisis dalam scope baru sehingga output `btab` memiliki block terpisah untuk body utama. Procedure, function, dan record juga membuat scope baru untuk parameter, variabel lokal, atau field.

### 3.6.2 Type Resolution

Tipe dasar diselesaikan melalui lookup predefined type. Tipe array menambahkan entri ke `atab`. Tipe record membuat block baru untuk field. Tipe subrange menyimpan base type dan batas bawah/atas. Tipe enumerated menyimpan daftar identifier dan mendaftarkan setiap enumerator sebagai konstanta.

### 3.6.3 Type Checking

Type checking dilakukan pada ekspresi dan statement. Operator aritmatika memerlukan tipe numerik. Operator Boolean memerlukan Boolean. Operator relasional memerlukan operand kompatibel dan menghasilkan Boolean. Assignment memerlukan target yang assignable dan value yang assignment-compatible terhadap target.

### 3.6.4 Procedure dan Function

Procedure dan function disimpan dalam `procSignatures`. Saat pemanggilan terjadi, analyzer mengecek jumlah argumen, tipe argumen, dan kebutuhan by-reference parameter. Function call pada expression menghasilkan tipe return function. Untuk function declaration, analyzer juga memeriksa apakah body function menjamin assignment ke nama function sebagai nilai return.

## 3.7 Format Output

Output program Milestone 3 terdiri dari lima bagian: `tab`, `btab`, `atab`, parse tree, dan decorated AST. Label object class mencakup `program`, `const`, `variable`, `type`, `procedure`, dan `function`. Decorated AST menampilkan anotasi seperti `tab_index`, `type`, `lev`, dan `block`.

## Bab 4

# Pengujian

### 4.1 Strategi Pengujian

Pengujian dilakukan dengan membangun program menggunakan `make -B`, menjalankan `arion_parser` pada input Milestone 3, lalu membandingkan output aktual dengan file output yang tersimpan pada direktori `test/milestone-3`. Command yang digunakan adalah sebagai berikut.

#### Command Pengujian

```
make -B
for i in 1 2 3 4 5 6 7 8; do
    ./arion_parser test/milestone-3/input- $\$i$ .txt output- $\$i$ .txt
    diff -q output- $\$i$ .txt test/milestone-3/output- $\$i$ .txt
done
```

Seluruh test case yang digunakan berhasil menghasilkan exit code 0 dan output yang konsisten dengan expected output. Laporan ini menampilkan lima kasus utama yang mewakili deklarasi, ekspresi, control flow, structured type, dan subprogram.

Tabel 4.1: Ringkasan Pengujian Milestone 3

| No | File Input                   | Fokus Pengujian                                                                                                    |
|----|------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1  | test/milestone-3/input-1.txt | Deklarasi variabel, assignment, ekspresi aritmatika, procedure call <code>writeln</code> , <code>tab/btab</code> . |
| 2  | test/milestone-3/input-2.txt | Operator aritmatika, <code>div</code> , <code>mod</code> , unary minus, <code>not</code> , dan kondisi Boolean.    |
| 3  | test/milestone-3/input-3.txt | Control flow: <code>while</code> , <code>for downto</code> , dan <code>repeat-until</code> .                       |
| 4  | test/milestone-3/input-4.txt | Structured type: record, array, field access, array access, dan case statement.                                    |
| 5  | test/milestone-3/input-5.txt | Procedure, function, parameter, local scope, recursive function call, dan function return.                         |

## 4.2 Detail Pengujian

### 4.2.1 Kasus 1: Program Dasar

#### Input

```
program Hello;  
  
var  
  a, b: integer;  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

#### Cuplikan Output

```
=== tab ===  
42      Hello      program      10      0      1      0      0      41  
43          a      variable      0      0      1      0      0      42  
44          b      variable      0      0      1      0      1      43  
  
=== btab ===  
0      44      0      0      2  
1      0      0      0      0  
  
=== Decorated AST ===  
Program('Hello') -> type:Unknown, lev:0, block:0  
  VarDecl('a') -> tab_index:43, type:Integer  
  VarDecl('b') -> tab_index:44, type:Integer  
  Block -> type:Unknown, lev:1, block:1  
    Assign  
    ProcCall('writeln')
```

Kasus ini menunjukkan bahwa program dasar dapat dianalisis secara semantik. Identifier program dicatat sebagai `program`, variabel `a` dan `b` masuk ke `tab`, block utama masuk ke `btabs`, dan decorated AST memiliki anotasi tipe serta block.

```
$ ./arion_parser test/milestone-3/input-1.txt test/milestone-3/output-1.txt

--- input-1.txt ---
program Hello;

var
  a, b: integer;
begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.

--- output-1.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer   type      0  0  1  0  0  0
34 Real      type      1  0  1  0  0  33
35 Char      type      2  0  1  0  0  34
36 Boolean   type      3  0  1  0  0  35
37 String    type      4  0  1  0  0  36
38 True      const     3  0  1  0  1  37
39 False     const     3  0  1  0  0  38
40 writeln   procedure 10  0  1  0  0  39
41 readln    procedure 10  0  1  0  0  40
42 Hello     program   10  0  1  0  0  41
43 a         variable 0  0  1  0  0  42
44 b         variable 0  0  1  0  1  43

=== btab ===
idx last lpar psze vsze
-----
0 44 0 0 2
1 0 0 0 0

=== atab ===
(empty)
...
```

Gambar 4.1: Bukti input dan output pengujian Kasus 1.

## 4.2.2 Kasus 2: Ekspresi dan Boolean

### Input

```
program Expressions;

var
  x, y, z: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2 + 1;
  z := -x mod 3;
  if not (x == 0) then
    writeln(y + z);
end.
```

### Cuplikan Output

```
Program('Expressions') -> type:Unknown, lev:0, block:0
VarDecl('x') -> type:Integer
VarDecl('y') -> type:Integer
VarDecl('z') -> type:Integer
Block -> lev:1, block:1
  Assign x := (10 + 5) - 3 * 2
  Assign y := x div 2 + 1
  Assign z := -x mod 3
  If -> condition type:Boolean
    ProcCall('writeln')
```

Kasus ini menguji inferensi tipe untuk operator aritmatika, integer division, modulo, unary minus, dan operator not. Kondisi pada if berhasil dianotasi sebagai Boolean.



```
$ ./arion_parser test/milestone-3/input-2.txt test/milestone-3/output-2.txt

--- input-2.txt ---
program Expressions;

var
  x, y, z: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2 + 1;
  z := -x mod 3;
  if not (x == 0) then
    writeln(y + z);
  end.

--- output-2.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer  type    0  0  1  0  0  0
34 Real    type    1  0  1  0  0  33
35 Char    type    2  0  1  0  0  34
36 Boolean  type    3  0  1  0  0  35
37 String   type    4  0  1  0  0  36
38 True     const   3  0  1  0  1  37
39 False    const   3  0  1  0  0  38
40 writeln  procedure 10  0  1  0  0  39
41 readln   procedure 10  0  1  0  0  40
42 Expressions program 10  0  1  0  0  41
43   x      variable  0  0  1  0  0  42
44   y      variable  0  0  1  0  1  43
45   z      variable  0  0  1  0  2  44

=== btab ===
idx last lpar psze vsze
-----
0 45 0 0 3
...
```

Gambar 4.2: Bukti input dan output pengujian Kasus 2.

### 4.2.3 Kasus 3: Control Flow

#### Input

```
program ControlFlow;

var
  i, sum: integer;

begin
  sum := 0;

  while sum < 100 do
  begin
    sum := sum + 1;
  end;

  for i := 10 downto 1 do
  begin
    sum := sum - i;
  end;

  repeat
    sum := sum + 5;
    i := i + 1;
  until sum >= 50;
end.
```

#### Cuplikan Output

```
Program('ControlFlow') -> type:Unknown, lev:0, block:0
VarDecl('i') -> type:Integer
VarDecl('sum') -> type:Integer
Block -> lev:1, block:1
  While -> condition type:Boolean
    Assign sum := sum + 1
  For('i', downto)
    Assign sum := sum - i
```

```
Repeat
  Assign sum := sum + 5
  Assign i := i + 1
  condition type:Boolean
```

Kasus ini memvalidasi bahwa semantic analyzer mampu menangani `while`, `for`, dan `repeat-until`. Ekspresi kondisi pada loop diuji agar menghasilkan tipe Boolean.

```
$ ./arion_parser test/milestone-3/input-3.txt test/milestone-3/output-3.txt

--- input-3.txt ---
program ControlFlow;

var
  i, sum: integer;

begin
  sum := 0;

  while sum < 100 do
    begin
      sum := sum + 1;
    end;

    for i := 10 downto 1 do
      begin
        sum := sum - i;
      end;

      repeat
        sum := sum + 5;
        i := i + 1;
      until sum >= 50;
    end.
  end.

--- output-3.txt (snippet) ---
=== tab ===
idx id      obj      type  ref  nrm  lev  adr  link
... (reserved words 0-32)
33 Integer   type    0    0    1    0    0    0
34 Real      type    1    0    1    0    0    33
35 Char      type    2    0    1    0    0    34
36 Boolean   type    3    0    1    0    0    35
37 String    type    4    0    1    0    0    36
38 True      const   3    0    1    0    1    37
39 False     const   3    0    1    0    0    38
...
```

Gambar 4.3: Bukti input dan output pengujian Kasus 3.

## 4.2.4 Kasus 4: Structured Type

### Input

```
program StructuredTypes;

type
  Point == record
    x, y: integer;
  end;

var
  p: Point;
  scores: array[1..5] of integer;
  grade: char;

begin
  p.x := 10;
  p.y := 20;
  scores[1] := p.x + p.y;

  grade := 'A';
  case grade of
    'A': writeln('Excellent');
    'B', 'C': writeln('Good');
  end;
end.
```

### Cuplikan Output

```
=== atab ===
idx  xtyp  etyp  eref  low  high  elsz  size
   0    7    0    0    1    5    1    5

Program('StructuredTypes') -> type:Unknown
TypeDecl('Point') -> type:Record
  RecordType
    VarDecl('x')
    VarDecl('y')
VarDecl('p') -> type:Record
VarDecl('scores') -> type:Array
VarDecl('grade') -> type:Char
Block
  FieldAccess(.x)
  FieldAccess(.y)
  ArrayAccess scores[1]
  Case
```

Kasus ini menguji record, array, akses field, akses indeks array, serta case statement. Informasi array masuk ke atab, sedangkan field record disimpan melalui block record.

```
$ ./arion_parser test/milestone-3/input-4.txt test/milestone-3/output-4.txt

--- input-4.txt ---
program StructuredTypes;

type
  Point == record
    x, y: integer;
  end;

var
  p: Point;
  scores: array[1..5] of integer;
  grade: char;

begin
  p.x := 10;
  p.y := 20;
  scores[1] := p.x + p.y;

  grade := 'A';
  case grade of
    'A': writeln('Excellent');
    'B', 'C': writeln('Good');
  end;
end.

--- output-4.txt (snippet) ---
=== tab ===
idx id      obj      type  ref  nrm  lev  adr  link
-----
... (reserved words 0-32)
33 Integer  type    0    0    1    0    0    0
34 Real    type    1    0    1    0    0    33
35 Char    type    2    0    1    0    0    34
36 Boolean type    3    0    1    0    0    35
37 String  type    4    0    1    0    0    36
38 True    const   3    0    1    0    1    37
39 False   const   3    0    1    0    0    38
...
```

Gambar 4.4: Bukti input dan output pengujian Kasus 4.

## 4.2.5 Kasus 5: Subprogram

### Input

```
program Subprograms;

var
  result: integer;

procedure PrintTwice(msg: string; count: integer);
var i: integer;
begin
```

```
for i := 1 to count do
begin
  writeln(msg);
end;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
  end;
end;

begin
  PrintTwice('hello', 3);
  result := Factorial(5);
  writeln(result);
end.
```

### Cuplikan Output

```
Program('Subprograms') -> type:Unknown
VarDecl('result') -> type:Integer
ProcedureDecl('PrintTwice') -> block:1
  Param('msg': string)
  Param('count': integer)
  VarDecl('i') -> type:Integer
  For('i', to)
    ProcCall('writeln')
FunctionDecl('Factorial', returns: 'integer') -> type:Integer, block:2
  Param('n': integer)
  If
    Assign Factorial := 1
    Assign Factorial := n * Factorial(n - 1)
Block -> block:3
  ProcCall('PrintTwice')
  FuncCall('Factorial') -> type:Integer
  ProcCall('writeln')
```

Kasus ini menunjukkan bahwa procedure dan function dicatat dalam symbol table, parameter diproses pada scope subprogram, pemanggilan recursive function dikenali sebagai FuncCall, dan function Factorial memiliki assignment ke nama function sebagai return value.

```
$ ./arion_parser test/milestone-3/input-5.txt test/milestone-3/output-5.txt

--- input-5.txt ---
program Subprograms;

var
  result: integer;

procedure PrintTwice(msg: string; count: integer);
var i: integer;
begin
  for i := 1 to count do
  begin
    writeln(msg);
  end;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
  end;
end;

begin
  PrintTwice('hello', 3);
--- output-5.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer  type    0  0  1  0  0  0
34 Real     type    1  0  1  0  0  33
35 Char     type    2  0  1  0  0  34
36 Boolean  type    3  0  1  0  0  35
37 String   type    4  0  1  0  0  36
38 True     const   3  0  1  0  1  37
...
```

Gambar 4.5: Bukti input dan output pengujian Kasus 5.

### 4.3 Pengujian Error Semantic

Selain kasus valid, program juga diuji secara manual terhadap input yang mengandung kesalahan semantik. Contoh kesalahan yang berhasil dideteksi adalah undeclared identifier, assignment dengan tipe tidak kompatibel, kondisi if non-Boolean, redeklarasasi identifier, index array dengan tipe salah, dan pemanggilan by-reference dengan literal.

```
Input:
program BadType;
var x: integer; s: string;
begin
  x := s;
end.

Output:
=== Semantic Errors ===
- Assignment types are incompatible
```

## Bab 5

# Penutup

### 5.1 Kesimpulan

Milestone 3 berhasil menambahkan tahap semantic analysis pada Arion Compiler. Program dapat menerima source code, token stream, atau parse tree, kemudian menghasilkan parse tree, AST, symbol table, dan decorated AST. Implementasi AST membuat representasi program menjadi lebih ringkas dibanding parse tree, sedangkan semantic analyzer menambahkan informasi tipe, scope, block, dan referensi symbol table.

Symbol table `tab`, `btab`, dan `atab` telah diimplementasikan untuk menyimpan identifier, block, dan array. Program juga telah mendukung predefined identifier, registrasi deklarasi, lookup identifier, pengecekan redeklarasi, type checking, assignment compatibility, procedure/function call, array access, record field access, serta validasi kondisi control flow.

Berdasarkan pengujian, program berhasil menangani kasus dasar, ekspresi, control flow, structured type, dan subprogram. Program juga mampu melaporkan beberapa semantic error secara informatif tanpa crash.

### 5.2 Saran

Pengembangan berikutnya dapat menambahkan analisis data-flow yang lebih lengkap untuk mendeteksi penggunaan variabel sebelum inisialisasi pada seluruh jalur eksekusi. Selain itu, pengecekan string length, kompatibilitas record anonymous, dan validasi cakupan nilai subrange pada ekspresi non-literal dapat diperluas agar semakin dekat dengan bahasa Pascal-like yang lengkap.

Untuk tahap selanjutnya, decorated AST yang sudah tersedia dapat dimanfaatkan sebagai dasar intermediate representation atau code generation. Pengujian juga dapat diperluas dengan kasus error yang lebih banyak, termasuk nested procedure/function, array bertingkat, record di dalam array, dan kombinasi scope yang lebih kompleks.

## Bab 6

# Lampiran

### 6.1 Tautan GitHub

- Repository: <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026>
- Halaman release: <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026/releases>

## 6.2 Tabel Kontribusi

| No | Nama Lengkap                | NIM      | Deskripsi Pekerjaan                                                                                                                                                                         | Kontribusi |
|----|-----------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1  | Jonathan Kris Wicaksono     | 13524023 | AST: merancang node hierarchy AST, Syntax-Directed Translation untuk konversi parse tree ke AST, dan AST formatter untuk output Decorated AST.                                              | 25%        |
| 2  | Vincent Rionarlie           | 13524031 | Semantic Analyzer: implementasi visit functions untuk setiap node, type checking dan compatibility rules, inferensi tipe, anotasi node AST, dan control flow validation.                    | 25%        |
| 3  | Muhammad Aufar Rizqi Kusuma | 13524061 | Symbol Table: implementasi <code>tab</code> , <code>btab</code> , <code>atab</code> , manajemen scope push/pop, registrasi dan lookup identifier, serta inisialisasi predefined identifier. | 25%        |
| 4  | Bryan Pratama Putra Hendra  | 13524067 | Laporan dan README: penulisan laporan lengkap dan README, penyusunan bagian teori, serta quality assurance.                                                                                 | 25%        |

## 6.3 Cara Menjalankan Program

```
make -B
make run INPUT=test/milestone-3/input-1.txt
make run-output INPUT=test/milestone-3/input-1.txt OUTPUT=output-m3.txt
```



# Bibliografi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2007.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 2nd edition, 1998.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 2nd edition, 2011.
- [4] Kenneth C. Louden. *Compiler Construction: Principles and Practice*. PWS Publishing Company, 1997.
- [5] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2010.
- [6] Niklaus Wirth. *Compiler Construction*. Addison-Wesley, 1996.